

Laboratory Research Problem 01 (LRP01)

CMSC 180: Introduction to Parallel Computing

Charles Andrei P. De los Reyes

CD-3L

February 4, 2026

Research Question 1

What do you think is the complexity of solving the MMT of a $n \times n$ square matrix X ? (hint: CMSC 142)

The complexity of solving the MMT for an $n \times n$ square matrix X is $O(n^2)$, which is known as quadratic complexity in CMSC 142. The `mmt` function uses a nested loop structure consisting of one outer loop and two inner loops. We can visualize the operations as $n \times (n + n) = 2n^2$. In Big-O notation, constants are dropped, leaving only n^2 . This results in a complexity of $O(n^2)$.

Data Table

Table 1: Runtime and Complexity Data

n	Run 1 (s)	Run 2 (s)	Run 3 (s)	Avg Runtime (s)	Comp. ($n = 100$)	Comp. ($\max(n)$)
100	0.000063	0.000063	0.000063	0.000063	0.000063	0.000172
200	0.000211	0.000211	0.000210	0.000211	0.000252	0.000689
300	0.000508	0.000515	0.000507	0.000510	0.000567	0.001551
400	0.000898	0.000900	0.001055	0.000951	0.001008	0.002757
500	0.001567	0.001529	0.001731	0.001609	0.001575	0.004308
600	0.002497	0.002361	0.002424	0.002427	0.002268	0.006204
700	0.004327	0.003544	0.004091	0.003987	0.003087	0.008445
800	0.005029	0.004774	0.004268	0.004690	0.004032	0.011029
900	0.007491	0.006897	0.007809	0.007399	0.005103	0.013959
1,000	0.012193	0.014034	0.009304	0.011844	0.006300	0.017234
2,000	0.035476	0.035147	0.039507	0.036710	0.025200	0.068935
4,000	0.174574	0.173671	0.176342	0.174862	0.100800	0.275741
8,000	0.800260	0.795991	0.821764	0.806005	0.403200	1.102964
16,000	3.545907	3.918605	3.587825	3.684112	1.612800	4.411851
20,000	6.937712	6.697515	7.045328	6.893518	2.520000	6.893518

Research Question 2

Were you able to run up to $n > 20,000$? If so, can you make it higher to 50,000 or even 100,000? If not, why do you think so and what do you need to do to make it so?

I was able to run up to $n = 46,000$ in 76.983459 seconds, but I was unable to run $n = 47,000$ or higher; the program simply stops when I attempt it. I believe the reason for this is limited physical RAM capacity. Although my laptop has 16GB of RAM, the OS and other background processes consume a significant portion of it. Consequently, allocating such a large contiguous block of memory exceeds my available physical memory. To run $n = 47,000$ and above, I would need to upgrade my hardware by installing more physical RAM.

Graph Analysis

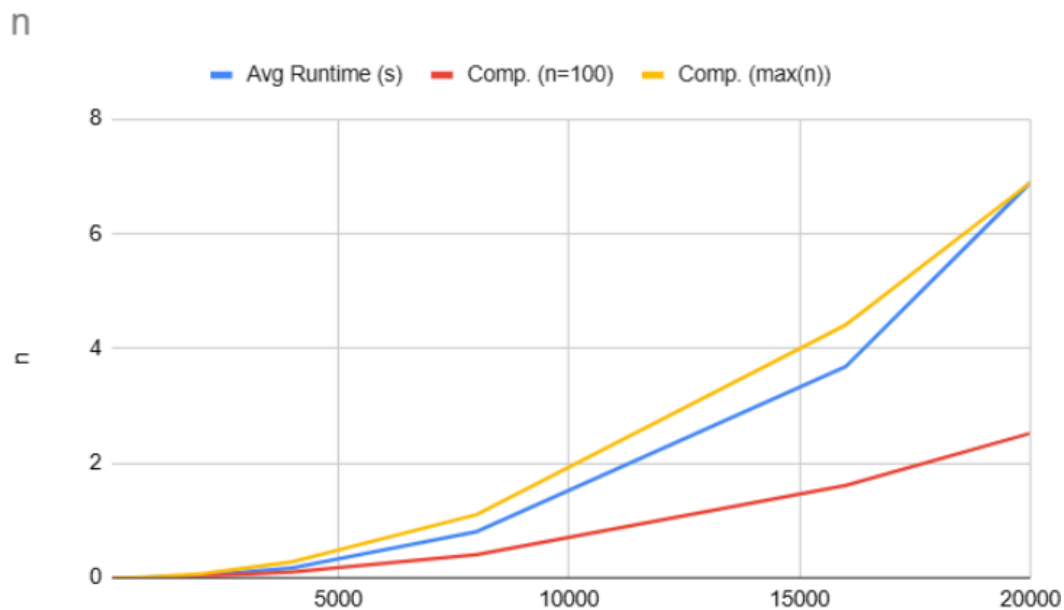


Figure 1: Plot of Matrix Size (n) vs. Average Runtime and Complexity

Research Question 3

Do the lines agree, at least in the form? If not, provide an explanation why so?

Yes, the lines agree in form. Based on the graph, the Average Runtime curve closely follows the Complexity (based on $\max(n)$) curve. Both exhibit a quadratic growth pattern. On the other hand, the Complexity (based on $n = 100$) line grows more slowly and remains below the measured runtime. Even though the lines grow at different rates, the overall shape and trend match, confirming that the results are consistent with quadratic complexity.

Research Question 4

Discuss ways on how we can make it better (lower average runtime) without using any extra processors or cores (notice that the word “ways” is in plural form).

Two effective ways to lower the average runtime without using any extra processors are improving memory access patterns and enabling compiler optimizations.

First, we can improve memory access patterns through techniques like Loop Tiling. Since C stores arrays in row-major order, our current column-based access forces the CPU to load a chunk of neighboring data but only use a single value from it before jumping to a distant memory address. This wastes the loaded data and forces the CPU to constantly fetch new chunks from the slow main RAM. Loop Tiling solves this problem by processing small blocks that fit entirely in the cache, ensuring we use loaded data before moving on.

Second, we can enable compiler optimization using flags like `-O3`. These flags allow the compiler to use Vectorization (SIMD - Single Instruction, Multiple Data). This utilizes special CPU registers to process multiple floating-point numbers in a single clock cycle, rather than processing them one by one. This reduces the instruction count and the execution time without any changes to the code.